

Entity resolution di problemi aziendali

Una pipeline ibrida retrieval + LLM per la deduplicazione
di segnalazioni tecniche estratte da conversazioni non strutturate

Francesco Lamberti

Corso di Laurea Magistrale in Ingegneria Informatica

Università degli Studi di Modena e Reggio Emilia

Tesina per l'esame di Business Intelligence

Docente: Prof. Domenico Beneventano

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Contesto applicativo	3
1.2	Obiettivo, contributo e struttura	3
2	Inquadramento teorico	4
2.1	Entity resolution	4
2.2	Blocking	4
2.3	Retrieval ibrido denso-sparso	4
2.4	LLM come matcher, clustering transitivo	4
2.5	Collocazione rispetto alla business intelligence	5
3	Il dataset	5
3.1	Origine e natura del dato	5
3.2	Vincoli di riservatezza	6
3.3	Unità di analisi	6
3.4	Perché le tecniche classiche di entity resolution non si applicano	6
4	Architettura della pipeline	7
4.1	Stadi 0 e 1: preprocessing e anonimizzazione	7
4.2	Stadio 2: estrazione delle segnalazioni	7
4.3	Stadio 3: entity resolution	8
4.4	Stadio 4: consolidamento	10
5	Discussione delle scelte progettuali	11
5.1	Il blocking come preconditione, non come ottimizzazione	11
5.2	Perché retrieval ibrido e non solo denso	11
5.3	Perché un LLM come classificatore	11
5.4	Chiusura transitiva: vantaggi e fragilità	12
5.5	Idempotenza, asimmetria, separazione dei ruoli	12
6	Risultati	13
6.1	Composizione del corpus ed efficacia del blocking	13
6.2	Esito della deduplicazione	14
6.3	Schema di uscita e statuto dei campi	14
7	Valutazione e limiti	15
7.1	Il problema della ground truth	15
7.2	Protocolli di valutazione praticabili	15
7.3	Limiti noti dell'approccio	15
8	Prospettive applicative e conclusioni	16
8.1	Dall'entità risolta all'indicatore	16
8.2	Sviluppi futuri	17
8.3	Conclusioni	17
A	Esempi di record	19
B	Configurazione sperimentale	19

1 Introduzione

1.1 Contesto applicativo

Nelle organizzazioni che gestiscono impianti logistici automatizzati una quota rilevante della conoscenza operativa non risiede in sistemi strutturati, ma nei canali di comunicazione informale. Il caso di studio affrontato in questa tesina riguarda un sistema di *Warehouse Management System* (WMS) in esercizio presso un'azienda manifatturiera, il cui supporto tecnico si è svolto per anni attraverso un gruppo di messaggistica istantanea condiviso tra il personale interno e i tecnici del fornitore.

In quel canale confluiscono, mescolati a comunicazioni di routine, tutti gli eventi rilevanti per l'esercizio dell'impianto: anomalie dei trasloelevatori, blocchi delle *pick station*, interruzioni della comunicazione tra sottosistemi, errori applicativi, interventi correttivi e *workaround*. È di fatto un registro degli incidenti, con la differenza sostanziale che non ha né schema, né identificatori, né vincoli di integrità: lo stesso guasto segnalato tre volte in tre anni diversi compare come tre conversazioni indipendenti, scritte da persone diverse, con lessico e livello di dettaglio diversi.

La conseguenza sul piano della *business intelligence* è immediata. Qualsiasi indicatore si voglia costruire sull'esercizio dell'impianto—frequenza dei guasti, ricorrenza per sottosistema, efficacia delle soluzioni adottate—presuppone di poter rispondere a una domanda preliminare, cioè *quante volte è accaduto lo stesso problema*. Contare le righe di una chat non risponde: conta le occorrenze testuali, non le entità. Il passaggio dall'una alle altre è esattamente il compito dell'*entity resolution*.

1.2 Obiettivo, contributo e struttura

L'obiettivo è progettare, implementare e discutere criticamente una pipeline che, partendo da conversazioni non strutturate, produca un insieme di *entità-problema* distinte, ciascuna descritta da una scheda sintetica a schema fisso e pronta per essere indicizzata in un sistema di *Retrieval-Augmented Generation* (RAG) a supporto del personale operativo.

Dal punto di vista della *data integration* classica il problema è un caso di *record linkage*. I record, però, non sono tuple con attributi confrontabili campo per campo, ma descrizioni in linguaggio naturale generate a loro volta da un modello linguistico: le tecniche tradizionali basate su distanze di stringa o su confronto attributo per attributo risultano quindi inapplicabili nella loro forma canonica. Il lavoro esplora come sostituirle mantenendo intatta l'architettura logica della pipeline.

Il contributo è principalmente **metodologico**. Non si propone un nuovo algoritmo, ma si documenta come i componenti canonici dell'*entity resolution*—blocking, comparison, classification, clustering, merging—siano stati reinterpretati in un contesto in cui i dati sono testo libero e il classificatore è un modello linguistico; per ogni scelta si esplicita la motivazione, l'alternativa scartata e il costo accettato. Una parte rilevante è dedicata ai limiti, e in particolare al problema della valutazione: per questo dataset non esiste una *ground truth* e la sua costruzione esaustiva sarebbe proibitiva, per ragioni analizzate quantitativamente nella Sezione 7.

Va precisato l'ambito del lavoro. Le prime due fasi—preprocessing e anonimizzazione—sono state sviluppate durante un tirocinio nell'ambito di un progetto RAG più ampio, e sono qui documentate perché costituiscono il presupposto indispensabile del resto: senza di esse il dato non sarebbe trattabile né utilizzabile al di fuori del perimetro aziendale. Il nucleo originale della tesina sono le fasi di *entity resolution* e di consolidamento, progettate ed eseguite specificamente per questo esame.

Il documento procede così: la Sezione 2 richiama i fondamenti teorici, la Sezione 3 descrive il dataset, la Sezione 4 presenta l'architettura stadio per stadio, la Sezione 5 discute le scelte progettuali, la Sezione 6 riporta i risultati, la Sezione 7 affronta valutazione e limiti, la Sezione 8 conclude.

2 Inquadramento teorico

2.1 Entity resolution

L'*entity resolution* (ER), nota anche come *record linkage* o *entity matching*, è il problema di identificare quali record si riferiscano alla stessa entità del mondo reale [2]. È un tema classico della *data integration*, studiato da decenni su dati tabellari, e la sua struttura si è dimostrata largamente indipendente dal dominio applicativo.

Dato un insieme $R = \{r_1, \dots, r_n\}$, si cerca una partizione di R in classi di equivalenza tali che due record appartengano alla stessa classe se e solo se descrivono la stessa entità. Il problema è intrinsecamente quadratico: la verifica esaustiva richiede

$$\binom{n}{2} = \frac{n(n-1)}{2} \quad (1)$$

confronti, costo che diventa rapidamente insostenibile al crescere di n , tanto più quando il singolo confronto è costoso.

Una pipeline di ER è convenzionalmente scomposta in quattro stadi [2, 6]: **blocking**, cioè riduzione dello spazio di ricerca a un insieme di coppie candidate; **comparison**, calcolo di una misura di similarità per ciascuna candidata; **classification**, decisione binaria o ternaria sulla coppia; **clustering e merging**, aggregazione delle decisioni a coppie in gruppi coerenti e produzione di un record rappresentativo, il cosiddetto *golden record*. Questa struttura è stata mantenuta integralmente nel presente lavoro; ciò che cambia è la realizzazione tecnica di ciascuno stadio.

2.2 Blocking

Il blocking è la risposta canonica al costo dell'Equazione (1): si raggruppano i record secondo una chiave poco costosa e si confrontano solo le coppie che condividono un blocco [12]. Ogni tecnica realizza un compromesso tra due quantità antagoniste: la **pair completeness**, frazione di coppie realmente duplicate che sopravvivono al blocking—una coppia scartata qui non è più recuperabile—e la **reduction ratio**, frazione di coppie eliminate rispetto al confronto esaustivo.

Nei dati strutturati il blocking usa chiavi esplicite (prefisso del cognome, codice postale, n -grammi). Nel testo non strutturato queste chiavi non esistono ed è naturale sostituirle con una ricerca per similarità: ciascun record viene usato come *query* contro gli altri e i primi k risultati ne costituiscono i candidati. È la strategia adottata qui.

2.3 Retrieval ibrido denso-sparso

Il blocking semantico è stato realizzato con una ricerca ibrida che combina due rappresentazioni complementari. La componente **sparsa** usa BM25 [15], che pesa i termini per frequenza nel documento e rarità nella collezione ed è efficace quando il segnale discriminante è lessicale; la componente **densa** usa BGE-M3 [1], modello di embedding multilingue che proietta il testo in uno spazio dove la prossimità riflette affinità semantica, ed è efficace quando lo stesso guasto è descritto con parole diverse. Le due falliscono in circostanze diverse e in larga misura scorrelate, e la loro combinazione per fusione dei ranghi [3] ha copertura superiore a ciascuna presa singolarmente—esattamente ciò che serve in uno stadio dove i falsi negativi sono irrecuperabili. La Sezione 5 argomenta la scelta sui dati specifici del dominio.

2.4 LLM come matcher, clustering transitivo

Comparison e classification sono tradizionalmente affidate a funzioni di similarità su stringhe combinate da un classificatore supervisionato, oppure a modelli neurali addestrati sul compito [10, 9]. Entrambe le strade presuppongono record con attributi allineati e, nel secondo caso, dati etichettati: nessuna delle due condizioni è qui soddisfatta. Un filone recente ha mostrato che i modelli linguistici di grandi dimensioni possono fungere direttamente da matcher in modalità *zero-shot*,

formulando il confronto come domanda in linguaggio naturale su una coppia di record [11, 13]. Il vantaggio è duplice: non serve addestramento e il criterio di equivalenza può essere espresso in modo dichiarativo nel prompt anziché codificato in una funzione di distanza. Lo svantaggio è che il costo per confronto cresce di diversi ordini di grandezza, il che rende il blocking non più un’ottimizzazione ma una precondizione di fattibilità.

Le decisioni prodotte riguardano coppie, mentre il risultato atteso è una partizione. Il criterio di aggregazione più semplice è la **chiusura transitiva**: se $A \equiv B$ e $B \equiv C$, allora i tre record appartengono allo stesso gruppo anche se la coppia (A, C) non è mai stata valutata. La struttura canonica per calcolarla incrementalmente è la *Union-Find*, con complessità ammortizzata quasi costante per operazione [16]. È una scelta attraente per semplicità, ma con un difetto strutturale documentato [5]: un singolo falso positivo può fondere due gruppi distinti, e l’errore si propaga in modo non locale.

2.5 Collocazione rispetto alla business intelligence

L’output della pipeline non è un fine in sé ma il substrato di un sistema di knowledge retrieval, secondo l’architettura RAG [8]. In questa prospettiva l’entity resolution svolge la funzione che in un processo ETL tradizionale spetta al *data cleaning*: senza deduplicazione una stessa informazione ricorrente satura i primi risultati del retrieval con copie quasi identiche, riducendo la diversità del contesto e con essa la qualità della risposta.

3 Il dataset

3.1 Origine e natura del dato

Il dato grezzo è l’esportazione testuale di un gruppo di messaggistica usato come canale di supporto tecnico per un sistema WMS, condiviso tra dipendenti dell’azienda utilizzatrice e tecnici del fornitore. Copre l’arco che va da metà 2021 a metà 2025, cioè dall’avviamento dell’impianto all’esercizio a regime.

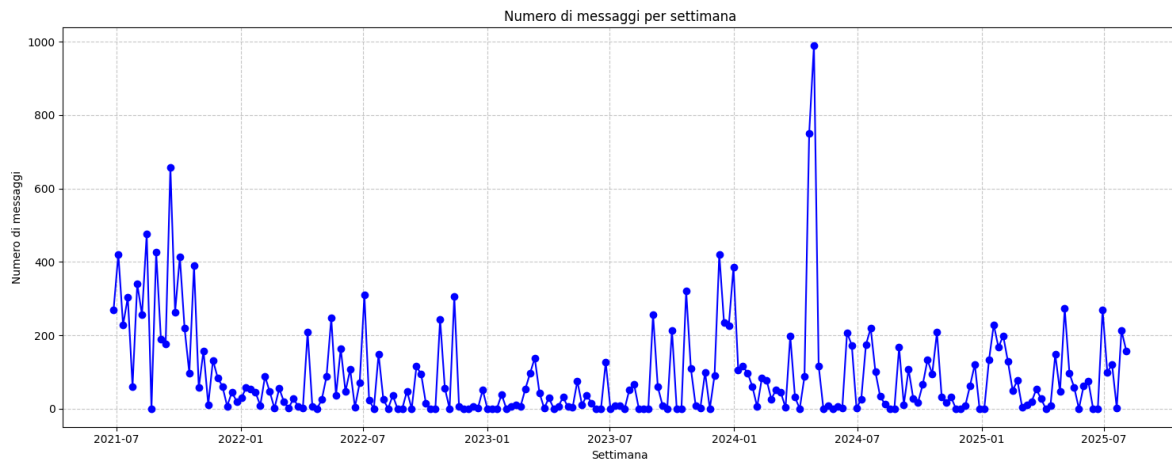


Figura 1: Distribuzione settimanale dei messaggi nel canale di supporto. Il picco iniziale corrisponde alla fase di avviamento dell’impianto; il picco isolato di inizio 2024 è riconducibile a un evento straordinario.

La Figura 1 restituisce la forma tipica di questo genere di corpus: il volume non è uniforme, con la fase di *ramp-up* del 2021 che concentra settimane oltre i 600 messaggi e gli anni successivi che alternano un regime di fondo a picchi isolati. La disomogeneità non è un difetto ma una proprietà informativa, e si riflette sulla distribuzione dei problemi per anno riportata nella Sezione 6.

Strutturalmente il corpus presenta tutte le caratteristiche del testo conversazionale: messaggi multiriga, allegati riferiti ma non contenuti, risposte implicite, un unico problema discusso a intermittenza per giorni, più problemi discussi in parallelo. Nessuna di queste caratteristiche è

compatibile con un’elaborazione diretta, perché il confine di un “problema” non coincide con alcun confine sintattico del testo.

3.2 Vincoli di riservatezza

Il corpus originale contiene dati che non possono circolare fuori dal perimetro aziendale: nomi di dipendenti e tecnici, numeri di telefono, ragioni sociali di clienti, codici identificativi interni (identificativi di pallet, *load unit*, riferimenti di dialogo tra sottosistemi) e, in alcuni scambi, importi economici.

Ne derivano due conseguenze progettuali. La prima è che l’anonimizzazione non è un accessorio ma uno stadio obbligato, collocato il più a monte possibile, così che tutti gli stadi successivi lavorino solo su dati già trattati. La seconda è che il repository non contiene il file di chat originale né i suoi derivati intermedi, che restano esclusi dal versionamento; sono invece pubblicati tutti gli artefatti a valle dell’anonimizzazione insieme al codice completo delle quattro fasi. Il limite di riproducibilità va dichiarato apertamente: le fasi 0 e 1 non sono rieseguibili da terzi, mentre lo sono integralmente le fasi 3 e 4—il nucleo della tesina—che prendono in ingresso esattamente i file pubblicati.

3.3 Unità di analisi

Dopo l’estrazione descritta nella Sezione 4 l’unità di analisi non è più il messaggio ma la **segnalazione di problema**: un blocco semi-strutturato che descrive un singolo evento tecnico secondo quattro campi (descrizione, causa identificata, soluzione o *workaround*, note), preceduto da data e titolo breve. Il corpus di lavoro conta 482 segnalazioni, con lunghezza media di 385 caratteri e mediana di 365, in un intervallo tra 178 e 811.

L’omogeneità dimensionale è una conseguenza voluta dell’estrazione guidata da schema: tutte le segnalazioni hanno la stessa forma e grosso modo la stessa densità informativa, il che rende il confronto a coppie molto più affidabile di quanto sarebbe stato confrontare frammenti di conversazione grezza di lunghezza arbitraria.

3.4 Perché le tecniche classiche di entity resolution non si applicano

L’armamentario standard dell’entity resolution, richiamato nella Sezione 2, non è utilizzabile su record di questo tipo: a ogni livello, dalla costruzione delle regole di confronto fino alla nozione stessa di equivalenza, l’impostazione classica presuppone qualcosa che in questo dominio manca.

L’approccio *rule-based* confronta i record campo per campo con funzioni di similarità tra stringhe—Levenshtein, Jaro-Winkler, Jaccard su n -grammi—combinare da regole scritte a mano, e richiede che i campi siano valori atomici e allineati; qui sono paragrafi in linguaggio naturale di lunghezza variabile. Applicare quelle distanze al testo intero non aiuta, perché tutto il corpus condivide lo stesso vocabolario tecnico ristretto: due segnalazioni qualsiasi hanno già una similarità di base elevata, mentre due occorrenze dello stesso guasto a quattro anni di distanza possono condividere pochissimo lessico. Manca anche una chiave di blocco, dato che l’unico campo strutturato è la data, qui addirittura controproducente: due occorrenze dello stesso problema ricorrente sono per definizione in date diverse. E i metodi supervisionati richiedono coppie annotate che, come discusso nella Sezione 7, per questo dataset non esistono.

La differenza più profonda è però un’altra. Nell’ER classica esiste un’entità nel mondo—una persona, un prodotto—di cui ogni record è un’osservazione rumorosa, e “è lo stesso oggetto” è per natura una relazione di equivalenza. Qui non esiste alcun oggetto con identità indipendente: due blocchi della stessa pick station a tre anni di distanza sono eventi distinti, che si *sceglie* di trattare come la stessa entità perché condividono causa e rimedio. L’equivalenza è quindi una convenzione di modellazione, e nulla garantisce a priori che sia transitiva—circostanza con conseguenze dirette sul clustering, discusse nella Sezione 5.

4 Architettura della pipeline

La pipeline si articola in cinque stadi sequenziali, ciascuno dei quali produce un artefatto persistente che costituisce l'input del successivo (Tabella 1).

Stadio	Artefatto prodotto	Motore	Ambiente
0. Preprocessing	ChatSSI_clean	regex deterministiche	CPU
1. Anonimizzazione	ChatSSI_clean_anon	Qwen3-32B (AWQ)	RTX 5090 / vLLM
2. Estrazione	202X_anon_singoli	Claude Opus 4.7	API
3. Entity resolution	problemi_deduplicati	BGE-M3 + BM25, Gemma 3 12B	locale / Qdrant + LM Studio
4. Consolidamento	singoli_dedup/*.md	Gemma 3 12B	locale / LM Studio

Tabella 1: Stadi della pipeline, artefatti e infrastruttura. Gli stadi 0 e 1 provengono dal lavoro di tirocinio; gli stadi 3 e 4 costituiscono il nucleo della tesina.

Materializzare su disco l'output di ogni stadio, anziché costruire un flusso unico in memoria, è una scelta deliberata: gli stadi che coinvolgono un LLM hanno tempi di esecuzione dell'ordine delle ore e non sono deterministici, e poterli rieseguire singolarmente, ispezionarne l'output e ripartire dall'ultimo punto valido è una necessità pratica prima ancora che una buona pratica di ingegneria del software.

4.1 Stadi 0 e 1: preprocessing e anonimizzazione

Il primo stadio applica trasformazioni puramente sintattiche. I numeri di telefono vengono intercettati per espressioni regolari—sia nel formato internazionale con separatori sia come sequenze non formattate—normalizzati alle sole cifre, così che occorrenze con spaziatura diversa dello stesso recapito collassino sulla stessa chiave, e sostituiti con un'etichetta di ruolo assegnata in base al prefisso internazionale: utenze italiane come personale interno, le altre come tecnici del fornitore. Si tratta di una *pseudonimizzazione* reversibile, e in questo stadio ciò è voluto, perché mantenere distinti gli interlocutori preserva la leggibilità dei botte e risposta. Una seconda trasformazione ricostruisce i messaggi multiriga: l'esportazione interrompe la riga a ogni ritorno a capo dell'autore, ma solo la prima riga di ciascun messaggio porta l'intestazione con data, ora e mittente, e riconoscendo questo marcatore è possibile riaccorpate le righe di continuazione in una tabella `datetime, sender, message`.

Le entità rimanenti—nomi propri, ragioni sociali, codici identificativi, importi—non hanno forma sintattica regolare e sono riconoscibili solo dal contesto. Lo stadio 1 le affida a Qwen3-32B [14] in quantizzazione AWQ, servito tramite vLLM [7] su GPU NVIDIA RTX 5090, processando il testo a finestra scorrevole di 50 righe. Il prompt elenca le categorie da sostituire (persone, aziende, codici identificativi, importi, ciascuna con la propria etichetta generica) e, altrettanto importante, quelle da preservare: timestamp, nomi di file allegati e soprattutto i termini tecnici di dominio.

Due aspetti meritano attenzione. Il primo è che ogni finestra viene anonimizzata in isolamento e senza mantenere alcuna mappa tra entità ed etichetta: tutte le persone diventano indistintamente **Persona**, tutte le aziende **Azienda**. L'anonimizzazione è quindi *irreversibile per costruzione*, perché non esiste da nessuna parte una corrispondenza salvata che consenta di invertirla—una garanzia strutturale, non procedurale, preferibile a una pseudonimizzazione coerente accompagnata da una tabella di traduzione che andrebbe a sua volta custodita. Il secondo è che l'esclusione esplicita dei termini tecnici è importante quanto l'elenco delle sostituzioni: senza di essa il modello tende a oscurare le sigle di sottosistema, che sono invece il segnale più discriminante disponibile a valle. Anonimizzare troppo avrebbe distrutto proprio l'informazione su cui si fonda l'entity resolution.

4.2 Stadio 2: estrazione delle segnalazioni

Il terzo stadio compie il salto di granularità decisivo, dalla sequenza di messaggi all'elenco di problemi. È un compito di *information extraction* guidato da schema, e il modo in cui è stato

risolto merita di essere raccontato a partire dal tentativo fallito.

Il primo approccio è stato condotto interamente in locale, riusando l'infrastruttura dello stadio precedente, cioè lo stesso Qwen3-32B in quantizzazione AWQ con la stessa strategia a finestra scorrevole. Non ha funzionato, e il motivo è strutturale più che dimensionale. L'anonimizzazione è un compito *locale*: decidere se una stringa sia un nome proprio non richiede di sapere che cosa sia successo duecento righe prima, e la finestra scorrevole è quindi una segmentazione innocua. L'estrazione dei problemi è invece un compito *globale*: una segnalazione si apre la mattina, viene ripresa nel pomeriggio, si chiude il giorno successivo, e nessuna finestra di cinquanta righe contiene abbastanza contesto per riconoscerne i confini. Segmentare il testo significa, in questo caso, distruggere proprio l'unità che si vuole estrarre—e il modello, vedendo frammenti, produceva problemi troncati, duplicati tra finestre adiacenti o inventati a partire da scambi di routine.

Serviva quindi un modello capace di tenere in un unico contesto un'annata intera di conversazioni, e con capacità di sintesi sufficienti a distinguere una segnalazione tecnica da una conversazione operativa. La scelta è caduta su Claude Opus 4.7, che dispone di una finestra di contesto nell'ordine del milione di token. Il corpus anonimizzato è stato suddiviso per anno e ogni annata elaborata in una singola invocazione, senza alcuna segmentazione.

Il ricorso a un servizio esterno per questo solo stadio è ammissibile precisamente grazie all'ordinamento della pipeline: il testo inviato è già passato per lo stadio 1 e non contiene più dati identificativi. È la ragione per cui l'anonimizzazione è stata collocata il più a monte possibile, e non subito prima della pubblicazione dei risultati.

Il prompt definisce il ruolo (analista tecnico di sistemi di warehouse management), lo schema di uscita a quattro campi, le regole di inclusione ed esclusione e—dettaglio operativamente rilevante—richiede un separatore esplicito `-%-` tra una segnalazione e l'altra, che consente di segmentare l'output programmaticamente senza dover interpretare la formattazione prodotta dal modello. Tra le regole, quella sull'accorpamento infragiornaliero (“se un problema si sviluppa in più momenti della stessa giornata, trattalo come singola voce”) costituisce già una prima forma di entity resolution, implicita e locale. Ciò che non può fare, e che motiva l'intero stadio successivo, è riconoscere la ricorrenza *tra* giornate e soprattutto *tra* anni, dato che ogni annata è stata elaborata in isolamento. L'esito sono cinque file, uno per anno, per un totale di 482 segnalazioni.

4.3 Stadio 3: entity resolution

Questo è il nucleo del lavoro: implementa i quattro momenti canonici descritti nella Sezione 2 ed è interamente eseguito in locale.

Indicizzazione. Le 482 segnalazioni vengono caricate in una collezione Qdrant—istanza eseguita in un container Docker locale—configurata in modalità ibrida: per ciascun record sono calcolate e memorizzate sia la rappresentazione densa di BGE-M3, sia quella sparsa BM25. A ogni nodo sono associati come metadati l'anno e il file di origine, che serviranno per distinguere i cluster intra-anno da quelli trasversali.

```

1 Settings.embed_model = HuggingFaceEmbedding(
2     model_name="BAAI/bge-m3", model_kwargs={"torch_dtype": "float16"})
3
4 vector_store = QdrantVectorStore(
5     collection_name=COLLECTION_NAME, client=qdrant,
6     enable_hybrid=True, # dense + sparse nella stessa collezione
7     fastembed_sparse_model="Qdrant/bm25")
8
9 nodes = [TextNode(text=p["text"],
10                  metadata={"year": p["year"], "source_file": p["source_file"]})
11           for p in problems]
```

Listing 1: Configurazione del vector store ibrido.



Figura 2: Proiezione bidimensionale dei 482 vettori densi BGE-M3 indicizzati nella collezione, ottenuta per riduzione di dimensionalità. I duplicati, che pure sono il 36,7% del corpus, non formano addensamenti riconoscibili. Una proiezione da oltre mille dimensioni a due distrugge gran parte della geometria originale, quindi l'immagine non prova l'assenza di struttura; è però coerente con quanto argomentato nella Sezione 3.4, cioè che non esista una soglia globale di distanza capace di separare i duplicati dai non duplicati.

Il testo dei nodi viene indicizzato *invariato*: questo consente di costruire una mappa diretta dal testo restituito dal retriever all'indice posizionale nella lista originale, e quindi di riportare i risultati della ricerca sulla struttura dati usata per il clustering senza gestire identificatori paralleli.

Blocking. Per ciascuna segnalazione si interroga l'indice in modalità ibrida usando come query il testo della segnalazione stessa, richiedendo i primi 6 risultati (`similarity_top_k`, `sparse_top_k` e `hybrid_top_k` tutti pari a 6). Il primo di essi è con altissima probabilità il record stesso e viene scartato, per cui il parametro corrisponde a 5 candidati effettivi per record.

Comparison e classification. Ogni coppia candidata è sottoposta al giudizio di Gemma 3 12B [4], in quantizzazione a 4 bit eseguito localmente tramite LM Studio. La macchina di esecuzione non è neutra rispetto alle scelte che seguono: gli stadi 3 e 4 girano su un personal computer con GPU AMD Radeon RX 6700 XT, dove LM Studio non dispone di CUDA e ricade sul backend Vulkan di `llama.cpp`. L'esecuzione si è rivelata sensibilmente più lenta e più fragile del servizio vLLM dello stadio 1, e ha condizionato l'intera configurazione a valle: la quantizzazione a 4 bit, un modello da 12 miliardi di parametri come limite pratico anziché come scelta ottimale, e un timeout di 300 secondi per chiamata, generoso al punto da risultare quasi inerte ma necessario per evitare che una singola risposta lenta interrompesse un ciclo di oltre un migliaio di invocazioni. Le due fasi, distinte nella pipeline classica, qui collassano in un unico atto: il modello non produce un punteggio da confrontare con una soglia, ma direttamente la decisione. Il prompt è volutamente minimale e contiene un solo elemento sostanziale, la definizione operativa di equivalenza.

Il modello è stato però eseguito con il ragionamento esplicito abilitato, e la ragione è la taglia ridotta: su un modello da 12 miliardi di parametri quantizzato a 4 bit, un verdetto emesso in un singolo token—senza che il confronto tra i tre elementi del criterio venga effettivamente svolto—non offre garanzie sufficienti. Con il ragionamento attivo il modello esamina esplicitamente i tre elementi—sistema coinvolto, causa e soluzione—prima di pronunciarsi, e solo la conclusione viene restituita nel formato binario richiesto dal prompt. È la scelta che spiega sia il budget di 4.096 token per chiamata, che serve alla catena di ragionamento e non alla risposta, sia la latenza di circa 24 secondi per confronto discussa nella Sezione 5: il costo dominante non è la risposta, ma la deliberazione che la precede.

```
Due problemi sono "simili" se condividono almeno due di questi tre elementi:
stesso sistema/componente coinvolto, stessa causa, stessa soluzione.
```

```
Problema A: {problem_a}
```

```
Problema B: {problem_b}
```

```
Sono lo stesso problema secondo questa definizione?
Rispondi SOLO con "SI" o "NO", senza spiegazioni.
```

Listing 2: Il prompt di matching.

La regola “almeno due elementi su tre” è la traduzione in linguaggio naturale di ciò che, in una pipeline tradizionale, sarebbe una soglia su una combinazione pesata di similarità per attributo; è una scelta di progetto sostanziale, discussa nella Sezione 5. Sul piano implementativo il parsing legge la sola conclusione, separata dalla catena di ragionamento, ed è conservativo: viene interpretata come affermativa solo una risposta che inizia per “SI”, e qualunque eccezione nella chiamata al modello è trattata come risposta negativa, così che un errore di infrastruttura non possa mai provocare una fusione spuria.

Clustering. Le decisioni positive sono accumulate in una struttura Union-Find con compressione di cammino. Il ciclo principale scorre le segnalazioni in ordine e, per ciascuna, valuta i candidati restituiti dal retriever applicando due filtri che precedono l’invocazione del modello.

```
1 uf = UnionFind(len(problems)); checked_pairs = set()
2
3 for i, problem in enumerate(problems):
4     for res in retriever.retrieve(problem["text"]):
5         j = text_to_idx.get(res.node.text)
6         if j is None or j == i: continue
7         if uf.find(i) == uf.find(j): continue # stesso cluster: transitività
8         pair_key = (min(i, j), max(i, j))
9         if pair_key in checked_pairs: continue # coppia simmetrica già valutata
10        checked_pairs.add(pair_key)
11        if are_same_problem(problem["text"], res.node.text):
12            uf.union(i, j)
```

Listing 3: Ciclo principale di entity resolution.

Il primo filtro sfrutta la transitività: se due record sono già nello stesso cluster, interrogare il modello sarebbe inutile perché l’esito non potrebbe cambiare la partizione. Il secondo sfrutta la simmetria attesa della relazione: poiché il blocking è mutuo—se B compare tra i candidati di A , spesso anche A compare tra quelli di B —la stessa coppia verrebbe altrimenti valutata due volte. L’effetto congiunto è quantificato nella Sezione 6: dei 2.410 confronti candidati generati dal blocking ne vengono eseguiti circa 1.700.

4.4 Stadio 4: consolidamento

L’ultimo stadio risolve il *merging*. Al termine dello stadio 3 un cluster di $k > 1$ occorrenze è ancora la semplice giustapposizione dei k testi originali: informativamente completo, ma ridondante e inadatto all’indicizzazione, perché un documento che descrive lo stesso guasto otto volte con parole diverse è un pessimo candidato per un sistema di retrieval. Per ogni cluster multiplo il modello riceve tutte le occorrenze, annotate con fonte e anno, e produce una scheda unica a quattro campi—descrizione generale, causa, soluzione consolidata e note—secondo un formato imposto rigidamente dal prompt. Il campo *Note* è progettualmente il più interessante, perché è l’unico che può contenere informazione emergente, non presente in nessuna occorrenza singola: l’evoluzione del problema nel tempo e l’eventuale stabilizzazione della soluzione.

In questo stadio il ragionamento esplicito è stato disattivato, al contrario di quanto fatto nello stadio 3. La ragione è che i due compiti hanno natura diversa: il matching è una decisione, e per giunta irreversibile, dove l’errore si propaga al clustering e va quindi pagato il costo di far deliberare il modello; il consolidamento è invece una sintesi guidata da un formato rigido, su materiale già selezionato, dove non c’è alcun verdetto da soppesare e la catena di ragionamento aggiungerebbe latenza senza migliorare l’uscita.

I cluster *singoletto* non vengono inviati al modello. Sarebbe stato possibile farlo per uniformare la formattazione, ma significherebbe 305 chiamate a un modello generativo su testi che non hanno bisogno di alcuna sintesi, con il rischio concreto di introdurre alterazioni in record corretti; su di essi si applica invece una pulizia deterministica che rimuove il prefisso di data, incorpora il titolo nella descrizione e normalizza i marcatori di formattazione. Il principio è generale e vale l'esplicitazione: *non si fa intervenire un modello generativo dove basta una trasformazione deterministica*. Ogni scheda è infine scritta in un file Markdown separato, formato che rende ciascuna entità direttamente indicizzabile come documento autonomo nel sistema RAG a valle.

5 Discussione delle scelte progettuali

Per ciascuna decisione rilevante si esplicita qui il problema affrontato, l'alternativa considerata e il costo accettato.

5.1 Il blocking come preconditione, non come ottimizzazione

Nelle pipeline di ER tradizionali il blocking è un'ottimizzazione: senza di esso il sistema funzionerebbe comunque, solo più lentamente. Qui non è così, e per mostrarlo basta la latenza misurata.

Lo stadio 3, nella configurazione descritta, ha richiesto una media di poco meno di 24 secondi per coppia, per un totale nell'ordine delle 11 ore sui circa 1.700 confronti effettivamente eseguiti. È una latenza altissima rispetto a una funzione di distanza tra stringhe, e si spiega con il concorso di quattro fattori: un prompt di circa 800 token, un modello da 12 miliardi di parametri, un backend Vulkan su GPU consumer priva di supporto CUDA e, soprattutto, il ragionamento esplicito abilitato in fase di generazione, che moltiplica i token prodotti per ogni singolo verdetto. Con $n = 482$ l'Equazione (1) dà 115.921 coppie: allo stesso ritmo, il confronto esaustivo avrebbe richiesto all'incirca 768 ore, cioè trentadue giorni di calcolo ininterrotto per una singola esecuzione—e la pipeline è stata rieseguita più volte durante lo sviluppo. Non è un costo elevato: è un costo che rende il compito semplicemente impossibile su quell'hardware.

L'adozione di un matcher costoso, quindi, *impone* il blocking. Le due scelte non sono indipendenti e non andrebbero valutate separatamente, perché la qualità del giudizio a coppie è stata acquistata al prezzo di rinunciare all'esaustività del confronto.

5.2 Perché retrieval ibrido e non solo denso

La scelta della modalità ibrida risponde alla struttura marcatamente bimodale del linguaggio del dominio. Una parte del lessico è altamente tecnica e sostanzialmente invariante—sigle di sottosistema, nomi di componenti, identificativi di stazioni e trasloelevatori—e su questi termini l'evidenza lessicale è fortissima: BM25 la cattura direttamente, mentre un embedding denso, che opera per compressione semantica, tende a rendere vicini codici che designano dispositivi fisicamente distinti perché la loro forma superficiale è quasi identica. Un'altra parte del lessico è invece del tutto libera, perché la descrizione discorsiva dell'anomalia riflette il modo di esprimersi di chi ha scritto il messaggio: lo stesso blocco può comparire come interruzione del flusso, fermo della stazione o attesa degli operatori, e qui è BM25 a essere cieco ed è l'embedding denso a fornire il segnale. Poiché i due meccanismi falliscono in circostanze diverse e in buona misura scorrelate, la loro unione ha copertura superiore a quella di ciascuno.

5.3 Perché un LLM come classificatore

L'alternativa naturale sarebbe stata una soglia sul punteggio di similarità restituito dal retriever, eliminando del tutto la chiamata al modello. È stata scartata per una ragione precisa: il punteggio misura *affinità testuale*, mentre il criterio richiesto è *identità causale*, e nel dominio in esame le due cose divergono sistematicamente. Due segnalazioni relative a pick station diverse, con cause

e soluzioni diverse, condividono gran parte del vocabolario e ottengono punteggi elevati: sono testualmente vicine e concettualmente distinte. Simmetricamente, due occorrenze dello stesso guasto a distanza di anni possono condividere pochissimo lessico ma avere identica causa radice. Nessuna soglia su un punteggio testuale separa correttamente i due casi, perché la quantità misurata non è quella che interessa.

Il modello linguistico consente invece di esprimere il criterio in modo *dichiarativo*: la regola “almeno due elementi su tre” descrive che cosa significhi equivalenza nel dominio, non come calcolarla. Ne derivano tre vantaggi—la regola è leggibile e discutibile con un esperto di dominio, è modificabile senza ricalibrare nulla, ed è documentabile come parte della definizione del problema anziché nascosta in una costante numerica—al prezzo del costo computazionale, della perdita di determinismo e dell'impossibilità di ottenere un punteggio graduato invece di una decisione secca.

La scelta di imporre una risposta binaria riguarda il formato dell'uscita, non il processo che la genera: il ragionamento esplicito è attivo, e il vincolo si applica soltanto alla conclusione, che deve essere interpretabile da un *if* senza ambiguità. È una scelta di robustezza prima che di efficienza, perché un esito in forma libera richiederebbe un parsing che introdurrebbe un'ulteriore fonte di errore proprio nel punto in cui la decisione diventa irreversibile. Il costo, però, è la rinuncia alla tracciabilità: la catena di ragionamento viene prodotta a ogni confronto ma non viene conservata, e a posteriori resta soltanto il verdetto, senza il motivo. È il limite più facilmente eliminabile dell'intera pipeline—basterebbe salvare la traccia accanto alla decisione, a costo nullo, per ottenere un registro ispezionabile di tutte le valutazioni. Resta invece il costo relativo alla confidenza: non esiste una zona grigia, e ogni coppia viene decisa in un verso o nell'altro anche quando il caso è genuinamente ambiguo. Una pipeline più matura prevedrebbe un terzo esito “incerto” con instradamento a revisione umana, secondo lo schema a tre classi—corrispondenza, non corrispondenza, caso dubbio—consolidato nella letteratura sul record linkage [2].

5.4 Chiusura transitiva: vantaggi e fragilità

La chiusura transitiva tramite Union-Find porta tre benefici concreti: garantisce che il risultato sia una partizione ben definita, riduce il numero di confronti saltando le coppie già nella stessa componente, e recupera coppie che il blocking avrebbe perso—se A e C non compaiono nelle rispettive liste di candidati ma entrambi sono collegati a B , la loro equivalenza viene comunque ricostruita. Quest'ultimo punto compensa parzialmente il limite principale del blocking, ed è il meccanismo che rende possibili i cluster di dimensione 7 e 8 osservati nei risultati, che nessun retriever con $k = 5$ avrebbe potuto costruire direttamente.

Lo stesso meccanismo è però la principale fragilità dell'approccio, perché l'equivalenza giudicata dal modello non è garantita essere transitiva: con un criterio “due su tre”, A può essere equivalente a B per sistema e causa, B equivalente a C per causa e soluzione, e A e C condividere solo la causa, dunque non essere equivalenti—ma la chiusura li fonde comunque. È il fenomeno noto come *effetto catena* [5]: il gruppo si estende lungo una successione di somiglianze locali, senza che nessuna coppia estrema sia mai stata verificata. Il rischio è proporzionale alla dimensione del cluster e va tenuto presente nella lettura dei risultati: i cluster piccoli sono ragionevolmente affidabili, quelli grandi vanno considerati ipotesi da verificare. Un approccio più conservativo—correlation clustering, o una soglia sul numero minimo di conferme interne al gruppo—avrebbe attenuato il problema, ma avrebbe richiesto la valutazione di tutte le coppie interne a ciascun cluster, reintroducendo per via indiretta il costo quadratico che il blocking serviva a evitare.

5.5 Idempotenza, asimmetria, separazione dei ruoli

Due dettagli implementativi apparentemente minori hanno implicazioni metodologiche. Il primo è l'insieme `checked_pairs`: la sua funzione evidente è il risparmio, quella meno evidente è la *coerenza*, perché la generazione non è deterministica e la stessa coppia valutata due volte potrebbe ricevere risposte diverse; senza memoria la partizione finale dipenderebbe dall'ordine di scansione in modo incontrollato. Il secondo è l'asimmetria dell'ordine degli argomenti: la coppia è sempre

presentata con il record corrente come Problema A e il candidato come Problema B, e nulla garantisce che il giudizio sia simmetrico—la letteratura sui LLM come annotatori documenta effetti di posizione anche su compiti in apparenza neutri. Il sistema assume implicitamente che lo sia; verificarlo su un campione sarebbe un'estensione a basso costo e alto valore diagnostico.

Un ultimo criterio ha guidato trasversalmente tutte le fasi: usare un modello linguistico solo dove il compito richiede effettivamente comprensione del contesto. L'applicazione più visibile è lo stadio 4, dove i 305 singoletti sono normalizzati per via deterministica; lo stesso principio si ritrova nello stadio 0 con i numeri di telefono e nella sostituzione finale del nome commerciale del sistema, realizzata con una singola `re.sub` a valle dell'anonimizzazione anziché delegata al prompt. Ne consegue che le schede finali arrivano da due percorsi distinti e portano etichette di campo diverse: come argomentato nella Sezione 6, la differenza è coerente con il diverso statuto dei due tipi di record, ma va tenuta presente da qualunque elaborazione a valle che assuma uno schema unico.

6 Risultati

6.1 Composizione del corpus ed efficacia del blocking

Anno	Segn.	%	Lungh. media
2021	164	34,0%	387
2022	68	14,1%	376
2023	76	15,8%	408
2024	113	23,4%	337
2025	61	12,7%	446
Tot.	482	100%	385

Tabella 2: Segnalazioni estratte per anno; la lunghezza media è il numero di caratteri per segnalazione. Il 2025 è un anno parziale.

Livello	Confronti	%
Esaustivo $\binom{482}{2}$	115.921	100,00%
Dopo blocking ($k = 5$)	2.410	2,08%
Dopo simmetria e transitività	≈ 1.700	1,47%

Tabella 3: Riduzione dello spazio di confronto. Il blocking elimina il 97,9% delle coppie, i filtri di simmetria e transitività circa il 29% delle residue.

La concentrazione sul 2021—oltre un terzo delle segnalazioni—riflette fedelmente il profilo di traffico della Figura 1, ed è attesa: la fase di avviamento di un impianto automatizzato è quella in cui emergono i problemi di configurazione e di integrazione tra sottosistemi. Il rialzo del 2024 corrisponde al picco visibile nel grafico.

La *reduction ratio* complessiva è del 98,5%, cioè un confronto ogni sessantotto. La latenza misurata è di poco meno di 24 secondi per confronto, il che colloca l'esecuzione dello stadio nell'ordine delle 11 ore; allo stesso ritmo il confronto esaustivo avrebbe richiesto circa 768 ore, cioè trentadue giorni ininterrotti. Anche i soli filtri di simmetria e transitività, che agiscono a valle del blocking, valgono circa 5 ore risparmiate sulle 16 che sarebbero servite a valutare tutte le 2.410 coppie candidate. È la differenza tra una pipeline eseguibile su hardware personale e una che non lo è. Va sottolineato che la pair completeness corrispondente non è nota e non può esserlo in assenza di ground truth: si sa quante coppie sono state eliminate, non quante coppie realmente duplicate siano state eliminate insieme a esse. Questa asimmetria—una metà del compromesso è misurabile, l'altra no—è il limite metodologico principale del lavoro. Le invocazioni complessive del modello nei due stadi di tesina sono circa 1.760: all'incirca 1.700 per il matching e 64 per il consolidamento. Il tempo del solo stadio 4 non è stato misurato, e non è derivabile dal dato precedente, perché le due invocazioni hanno profili di costo diversi: il consolidamento gira senza ragionamento esplicito, ma su prompt che contengono l'intero cluster e con una generazione di alcune centinaia di token. Resta valida l'unica osservazione ricavabile a rigore, cioè che 64 chiamate sono meno del 4% delle invocazioni totali: qualunque sia la loro durata unitaria, il collo di bottiglia della pipeline sta nel decidere, non nel fondere.

6.2 Esito della deduplicazione

Delle 482 segnalazioni in ingresso la pipeline produce 369 entità distinte, con una riduzione del 23,4%.

Dim.	Cluster	Segnalazioni	%
1	305	305	63,3%
2	36	72	14,9%
3	16	48	10,0%
4	8	32	6,6%
5	2	10	2,1%
7	1	7	1,5%
8	1	8	1,7%
Tot.	369	482	100%

Tabella 4: Distribuzione delle dimensioni dei cluster; percentuali arrotondate al decimo.

Tipo di cluster	N.	%
Occorrenze nello stesso anno	44	68,8%
Occorrenze su più anni	20	31,2%
Totale cluster multipli	64	100%

Tabella 5: Distribuzione temporale delle occorrenze nei 64 cluster di duplicati.

I 64 cluster non singoletto coinvolgono 177 segnalazioni, pari al 36,7% del corpus. La distribuzione ha l'andamento fortemente asimmetrico tipico dei fenomeni di ricorrenza: la maggioranza dei problemi si è verificata una sola volta, mentre un piccolo insieme di guasti si ripete con frequenza elevata. I due cluster maggiori, di 7 e 8 occorrenze, sono per definizione i più interessanti dal punto di vista della business intelligence—sono i problemi cronici dell'impianto—e, per la fragilità della chiusura transitiva discussa nella Sezione 5, anche quelli che meriterebbero per primi una verifica manuale.

Quasi un terzo dei duplicati (Tabella 5) collega segnalazioni di annate diverse. Il dato è significativo per due ragioni. La prima è di validazione interna: poiché lo stadio di estrazione ha elaborato ogni anno in isolamento, questi collegamenti non potevano essere prodotti prima dell'entity resolution, e costituiscono quindi informazione genuinamente nuova generata dalla pipeline. La seconda è sostanziale: un problema che riemerge a distanza di anni è quasi per definizione un problema mai risolto alla radice ma solo tamponato, ed è precisamente la classe di anomalie su cui un sistema di supporto alle decisioni dovrebbe attirare l'attenzione. Il caso più marcato è un cluster di 7 occorrenze relativo alla comunicazione tra il WMS e il sottosistema di movimentazione, con cinque interventi ravvicinati nel 2021 e due riemersioni nel 2023 e nel 2025.

6.3 Schema di uscita e statuto dei campi

L'output è costituito da 369 file Markdown indipendenti, ciascuno con una scheda a quattro campi. Le etichette non sono però identiche nei due percorsi di produzione: **Descrizione generale** e **Note** compaiono in tutte le 369 schede, mentre il campo relativo alla causa è etichettato **Causa identificata** nelle 305 schede prodotte dalla pulizia deterministica e **Causa** nelle 64 consolidate dal modello, e allo stesso modo **Soluzione / Workaround** si contrappone a **Soluzione / Workaround consolidato**.

La differenza riflette lo statuto diverso dei due tipi di record. In una scheda singoletto la causa è quella emersa dalla specifica conversazione, tanto che molte schede riportano esplicitamente “Non esplicitata”, e la soluzione è l'intervento applicato in quell'occasione. L'etichetta **Causa identificata** registra appunto un'attribuzione puntuale, riferita a un singolo episodio e verificabile nella fonte da cui la scheda proviene. In una scheda consolidata, invece, la causa non è identificata in una singola conversazione ma sintetizzata su più occorrenze, ed è quindi necessariamente più generica; e la soluzione non è un singolo intervento ma l'unione di ciò che veniva fatto ogni volta, da cui l'aggettivo **consolidato**. Chiamare “causa identificata” il risultato di una sintesi su sette segnalazioni sarebbe una sovra-attribuzione: si asserirebbe un accertamento puntuale dove esiste solo una regolarità osservata.

7 Valutazione e limiti

7.1 Il problema della ground truth

Il limite più rilevante di questo lavoro è l'assenza di una valutazione quantitativa dell'accuratezza: non esiste, per questo dataset, un insieme di riferimento annotato che indichi quali coppie siano effettivamente duplicate, e di conseguenza non è possibile calcolare precision, recall o F_1 né sul matching a coppie né sulla partizione finale.

L'assenza non è una dimenticanza ma una conseguenza strutturale del problema. Costruire una ground truth esaustiva significherebbe decidere manualmente, per ciascuna delle 115.921 coppie possibili, se le due segnalazioni descrivano lo stesso problema; anche ipotizzando ottimisticamente dieci secondi per giudizio, l'annotazione richiederebbe circa 322 ore di lavoro di un annotatore esperto, cifra fuori scala rispetto a una tesina e comunque sproporzionata per un corpus di 482 record. Il punto è generale ed è la ragione per cui l'entity resolution è difficile da valutare: il costo dell'annotazione cresce quadraticamente con la dimensione del corpus, mentre il suo valore informativo cresce molto più lentamente, dato che la stragrande maggioranza delle coppie è banalmente negativa.

7.2 Protocolli di valutazione praticabili

Che l'annotazione esaustiva sia impraticabile non implica che non sia possibile alcuna valutazione. Le strategie seguenti costituiscono la prima estensione naturale del lavoro.

Annotazione delle sole coppie candidate. Le coppie effettivamente valutate sono all'incirca 1.700, meno dell'1,5% del totale; annotarle tutte richiederebbe circa cinque ore e permetterebbe di calcolare precision e recall del *matcher* sulle coppie che il blocking gli ha sottoposto. Resterebbe non misurabile la pair completeness del blocking, ma si otterrebbe una misura esatta della componente più critica del sistema. È, con ampio margine, l'estensione con il miglior rapporto tra costo e valore informativo.

Campionamento stratificato per la precision. Per stimare la precision della partizione basta un campione: estraendo casualmente 50 dei 64 cluster multipli e verificando manualmente ciascuna fusione si otterrebbe una stima con intervallo di confidenza calcolabile. La stratificazione per dimensione è importante, perché il rischio di fusione spuria cresce con la dimensione del cluster e un campione uniforme sotto-rappresenterebbe proprio i casi più a rischio.

Pooling per la stima del recall. Il recall è strutturalmente più difficile da stimare, perché richiederebbe di conoscere i duplicati che il sistema non ha trovato. La tecnica standard è il *pooling* [17]: si eseguono più configurazioni sullo stesso corpus—variando k , usando solo la componente densa, usando solo BM25—e si annota l'unione dei risultati, usandola come approssimazione della verità. Il recall così stimato non è assoluto, ma consente il confronto relativo tra configurazioni, che è spesso ciò che serve per decidere.

Verifica di simmetria e stabilità. Due diagnostiche non richiedono annotazione manuale e misurano la coerenza interna anziché la correttezza: rivalutare un campione di coppie invertendo l'ordine degli argomenti, e rieseguire lo stesso confronto più volte a parità di input. Entrambe sono automatizzabili e forniscono un limite superiore all'affidabilità ottenibile: se il modello non è coerente con sé stesso, nessuna metrica di accuratezza può superare quella soglia.

7.3 Limiti noti dell'approccio

Recall limitato dal blocking. Con $k = 5$ candidati per record, un cluster di dimensione superiore a 6 non può essere costruito per via diretta ma solo attraverso la transitività, ed è infatti così che esistono i cluster di 7 e 8 elementi osservati. Ne segue che il sistema è strutturalmente

più debole proprio sui problemi più ricorrenti, che sono anche i più interessanti dal punto di vista applicativo. Un k adattivo, crescente per i record già appartenenti a cluster ampi, sarebbe un correttivo mirato e di costo contenuto.

Propagazione degli errori tra stadi. La pipeline è strettamente sequenziale e ogni stadio assume corretto l'output del precedente: un problema non riconosciuto in fase di estrazione non esiste per gli stadi successivi. Poiché l'estrazione è essa stessa affidata a un modello generativo, gli errori dell'entity resolution non sono separabili da quelli dell'estrazione, e una valutazione della sola fase 3 sovrastimerebbe la qualità del sistema complessivo.

Anonimizzazione non verificata, e residui nei prompt. Lo stadio di anonimizzazione è affidato a un modello linguistico e il suo output non è stato sottoposto a verifica sistematica; nella protezione dei dati un singolo falso negativo è sufficiente a compromettere la garanzia, e una pipeline destinata a uso produttivo dovrebbe affiancare al modello un controllo deterministico a valle—ricerca di pattern residui, confronto con liste di entità note—che non dipenda dalla stessa tecnologia che si intende verificare. A conferma, l'ispezione dei file estratti mostra che il nome commerciale del sistema WMS, sostituito con un termine generico durante l'anonimizzazione, ricompare nei file dello stadio 2, perché i prompt di estrazione lo nominano esplicitamente per contestualizzare il compito. L'informazione è di per sé poco sensibile, ma l'episodio illustra un rischio generale: in una pipeline in cui i prompt contengono informazioni di contesto, l'anonimizzazione del dato non garantisce l'anonimizzazione del risultato.

Sensibilità alla formulazione del criterio. L'intero comportamento del matcher dipende da una singola frase di prompt: non è stata condotta alcuna analisi di sensibilità rispetto a formulazioni alternative del criterio “due su tre”, né rispetto a criteri più restrittivi o più permissivi. Poiché quella frase svolge il ruolo che in una pipeline tradizionale spetterebbe a una soglia calibrata sui dati, la sua mancata validazione equivale ad adottare una soglia scelta a priori e mai verificata.

8 Prospettive applicative e conclusioni

8.1 Dall'entità risolta all'indicatore

L'output della pipeline abilita analisi che sul dato grezzo non sarebbero nemmeno definibili. Il punto non è che siano difficili da calcolare: è che non hanno significato finché l'entità-problema non esiste come oggetto distinto. La **frequenza di ricorrenza** diventa calcolabile perché un'entità raccoglie tutte le sue occorrenze, e i due cluster maggiori identificano immediatamente i guasti cronici dell'impianto—informazione che sul corpus originale richiederebbe la lettura integrale di cinque anni di conversazioni. La **persistenza temporale**, cioè i 20 cluster con occorrenze su più anni, individua i problemi mai risolti alla radice, che sono la classe su cui un intervento strutturale ha il ritorno maggiore. Il campo *Note* delle schede consolidate, l'unico a contenere informazione emergente dalla fusione, registra quando la soluzione è cambiata tra un'occorrenza e l'altra e fornisce un tracciato qualitativo dell'**efficacia degli interventi** nel tempo.

Il beneficio più immediato resta comunque quello per cui la pipeline è stata concepita, cioè il sistema RAG a valle. In un indice contenente otto descrizioni quasi identiche dello stesso guasto, un'interrogazione su quel tema restituisce otto varianti della stessa risposta, saturando il contesto e impedendo al modello generativo di accedere a informazione complementare; dopo la deduplicazione la stessa interrogazione restituisce una scheda consolidata—che include già la storia delle occorrenze e l'evoluzione della soluzione—e quattro documenti distinti su aspetti correlati. In questa prospettiva la deduplicazione non è una compressione del dato ma un aumento della densità informativa per unità di contesto recuperato.

8.2 Sviluppi futuri

Sul piano della **valutazione** l'intervento prioritario è l'annotazione manuale delle circa 1.700 coppie effettivamente valutate, che ha un costo di poche ore e trasformerebbe il lavoro da descrittivo a misurato; a seguire, le diagnostiche di simmetria e stabilità, interamente automatizzabili. Sul piano dell'**algoritmo** tre interventi sono mirati ai limiti identificati: un valore di k adattivo per i record appartenenti a cluster ampi, l'introduzione di un terzo esito "incerto" con instradamento a revisione umana, e la sostituzione della chiusura transitiva pura con un criterio che richieda un numero minimo di conferme interne al cluster. Sul piano dell'**ingegnerizzazione** un campo esplicito che dichiari il percorso di provenienza di ciascuna scheda renderebbe la distinzione descritta nella Sezione 6 leggibile da un parser, sbloccando l'uso dell'output in contesti a schema fisso senza dover rinominare i campi; più in generale, la pipeline è oggi una sequenza di notebook eseguiti manualmente, adeguata a un lavoro esplorativo ma non a un aggiornamento periodico, e l'incrementalità—aggiungere nuove segnalazioni a una partizione già calcolata senza rieseguire tutto—è il requisito principale per un eventuale passaggio in esercizio, che la struttura Union-Find già adottata è per natura adatta a supportare.

8.3 Conclusioni

Il lavoro ha mostrato che l'architettura canonica dell'entity resolution resta valida quando i record sono testo libero anziché tuple strutturate, purché ciascuno stadio venga reimplementato con strumenti adeguati alla natura del dato: retrieval ibrido denso-sparso al posto delle chiavi di blocco, modello linguistico interrogato in modalità binaria al posto delle distanze di stringa e del classificatore supervisionato, chiusura transitiva su Union-Find per l'aggregazione, secondo passaggio generativo per la fusione.

Sui 482 record del corpus la pipeline ha prodotto 369 entità distinte, individuando 64 gruppi di ricorrenza di cui 20 trasversali a più anni—collegamenti che nessuno stadio precedente avrebbe potuto produrre, avendo elaborato ogni annata in isolamento. Il blocking ha ridotto lo spazio di confronto del 98,5%, portando l'esecuzione nell'ordine delle 11 ore su hardware personale, contro le oltre settecento—più di un mese di calcolo ininterrotto—che la formulazione esaustiva avrebbe richiesto allo stesso ritmo.

Il risultato metodologicamente più significativo, però, non riguarda i numeri prodotti ma il compromesso che li ha generati. L'adozione di un classificatore semanticamente ricco ma costoso rende il blocking una preconditione di fattibilità anziché un'ottimizzazione, e sposta di conseguenza il baricentro del rischio: il sistema non sbaglia tanto sulle coppie che valuta, quanto su quelle che non arriva mai a valutare. È anche il motivo per cui la valutazione dell'approccio è così difficile, dato che la componente più critica è quella la cui misura richiederebbe esattamente il confronto esaustivo che si è voluto evitare. Riconoscere questa circolarità, più che risolverla, è forse il contributo più onesto che un lavoro di questa scala possa offrire.

Riferimenti bibliografici

- [1] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 2024.
- [2] Peter Christen. *Data Matching: Concepts and Techniques for Record Linkage, Entity Resolution, and Duplicate Detection*. Data-Centric Systems and Applications. Springer, 2012.
- [3] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.
- [4] Gemma Team. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.
- [5] Oktie Hassanzadeh, Fei Chiang, Hyun Chul Lee, and Renée J. Miller. Framework for evaluating clustering algorithms in duplicate detection. *Proceedings of the VLDB Endowment*, 2(1):1282–1293, 2009.
- [6] Hanna Köpcke and Erhard Rahm. Frameworks for entity matching: A comparison. *Data & Knowledge Engineering*, 69(2):197–210, 2010.
- [7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
- [9] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep entity matching with pre-trained language models. *Proceedings of the VLDB Endowment*, 14(1):50–60, 2020.
- [10] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. Deep learning for entity matching: A design space exploration. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 19–34, 2018.
- [11] Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. Can foundation models wrangle your data? *Proceedings of the VLDB Endowment*, 16(4):738–746, 2022.
- [12] George Papadakis, Dimitrios Skoutas, Emmanouil Thanos, and Themis Palpanas. Blocking and filtering techniques for entity resolution: A survey. *ACM Computing Surveys*, 53(2):1–42, 2020.
- [13] Ralph Peeters and Christian Bizer. Using chatgpt for entity matching. In *Advances in Databases and Information Systems (ADBIS)*, pages 221–230, 2023.
- [14] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [15] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [16] Robert Endre Tarjan. Efficiency of a good but not linear set union algorithm. *Journal of the ACM*, 22(2):215–225, 1975.
- [17] Ellen M. Voorhees. Variations in relevance judgments and the measurement of retrieval effectiveness. *Information Processing & Management*, 36(5):697–716, 2000.

A Esempi di record

Gli esempi provengono dal corpus già anonimizzato e illustrano i due percorsi di produzione delle schede finali, con la conseguente differenza nelle etichette di campo discussa nella Sezione 6.

Descrizione generale: Problemi di comunicazione tra i sistemi WMS e MFS che causano il blocco della movimentazione dei pallet (TP0) verso le pick stations, con conseguente fermo delle attività di picking e inattività degli operatori. Causa: Malfunzionamenti nel flusso dati tra WMS e MFS, spesso legati a residui di deploy, riavvii del sistema o problemi di refresh della comunicazione. Soluzione / Workaround consolidato: Riavvio dei sistemi (MFS), esecuzione di deploy correttivi, fix manuali delle linee e applicazione di workaround per ripristinare la comunicazione. Note: Il problema è persistente dal 2021 al 2025; sebbene siano stati applicati patch e workaround frequenti, l'issue rimane critica per l'automazione e non è stata ancora risolta definitivamente in modo permanente.

Listing 4: Scheda consolidata da un cluster di 7 occorrenze (2021, 2023, 2025). Il campo *Note* contiene informazione emergente dalla fusione, assente in ogni occorrenza singola.

Descrizione generale: Pick Station 7 non conferma "full pallet", SRM8 bloccato. Impossibile confermare "full pallet" alla stazione 7. SRM8 bloccato. Pick stations 3, 4, 6, 12 in attesa. Impatto: blocco multiplo. Causa identificata: Pallet in HBW08 (aisle con crane bloccato). Il pulsante "full pallet" non funziona se una linea dell'OBD è in stato "New". Soluzione / Workaround: Finalizzata la linea manualmente. "Full pallet" ha funzionato. Ticket aperto per la fix. Note: Bug: "Pallet full button not working if line in status New".

Listing 5: Scheda singoletto, prodotta dal percorso deterministico: si notino le etichette di campo diverse dall'esempio precedente.

B Configurazione sperimentale

Componente	Modello	Esecuzione
Anonimizzazione (stadio 1)	Qwen3-32B, quantizzazione AWQ	vLLM su NVIDIA RTX 5090
Estrazione (stadio 2)	Claude Opus 4.7	API, un'annata per invocazione
Embedding denso / sparso	BAAI/bge-m3 (fp16), Qdrant/bm25	locale, via FastEmbed
Backend di inferenza (st. 3-4)	llama.cpp, backend Vulkan	nessun supporto CUDA sulla macchina
Vector store	Qdrant, collezione ibrida	container Docker locale
Matching e consolidamento	Gemma 3 12B, quantizzazione 4 bit	LM Studio su Radeon RX 6700 XT
<i>Iperparametri</i>		
Finestra anonimizzazione	50 righe	senza sovrapposizione
Temperatura	0,1 (anonimizz.) / 0,2 (matching)	bassa per favorire la riproducibilità
similarity/sparse/hybrid_top_k	6	5 candidati effettivi più il record stesso
Context window (matching)	16.384 token	ampiamente sufficiente per due record
max_tokens	4.096 (matching) / 600 (consolid.)	budget per la catena di ragionamento
Ragionamento esplicito	attivo (st. 3), disattivo (st. 4)	deliberazione solo dove si decide
Timeout per chiamata	300 s	marginale per l'inferenza locale
Tempo di esecuzione stadio 3	≈ 11 h per ≈ 1.700 confronti	≈ 24 s per confronto, misurato
Tempo di esecuzione stadio 4	non misurato	64 chiamate, 4% del totale

Tabella 6: Modelli, infrastruttura e iperparametri della pipeline.

Riproducibilità. Il codice è organizzato in quattro notebook nella cartella `Application` del repository, da eseguire nell'ordine `messagesPreprocessing`, `anonymize`, `entity_resolution`, `entity_resolution`. Le dipendenze principali sono `llama-index 0.14`, `qdrant-client 1.17`, `sentence-transformers 6.0` e `fastembed 0.8`; l'elenco completo è in `requirements.txt`. Come discusso nella Sezione 3,

le fasi 0 e 1 non sono rieseguibili da terzi non essendo il corpus originale versionato; le fasi 3 e 4 sono invece integralmente riproducibili a partire dai file `Documents/202X_anon_singoli.txt` pubblicati, previa disponibilità di un'istanza Qdrant e di un endpoint OpenAI-compatibile che serva il modello di matching. Trattandosi di generazione a temperatura non nulla, l'esatta partizione ottenuta può variare tra esecuzioni.